Gomi — Auditoría técnica front y back

Nov 5, 2026 · @Freddy

El problema de fondo no son parches sueltos: el mapa y la creación de lugares tienen varias fuentes de verdad para el mismo dato, y cada fix toca solo una. Hay un riesgo de seguridad en el backend que conviene corregir antes que cualquier refactor.

Alcance: frontend (~36k líneas TS/TSX) y backend (~12k líneas JS). Análisis automático de imports, valores quemados y SQL, más lectura completa del mapa y de los servicios de lugares. Las pantallas del admin no se leyeron línea por línea.

Prioridades

Once hallazgos, ordenados por severidad. Los cinco primeros explican la mayoría de bugs repetidos y deberían resolverse antes de construir B2B encima.

#	Hallazgo	Dónde	Impacto	Severidad
1	Al aprobar un change request, las propiedades se interpolan como texto en SQL	back `place-change-request.service.js:363-375`	El endpoint de change requests es público: un apóstrofe rompe la aprobación y abre inyección SQL	Crítica
2	Las sugerencias del formulario siempre van al pack 1 (Quito)	front `collectFormData.ts:35`, `useFormSchema.ts:22`; el back no valida coordenadas contra el polígono	Un lugar sugerido en Cuenca se guarda en Quito, con el formulario de Quito	Alta
3	El mapa no tiene un dueño único del estado de zona ni de la cámara	`ViewMap.tsx`, `LocationMap.tsx`	23 de 35 commits del mapa desde agosto son fixes; `LocationMap` cambió 17 veces	Alta

#	Hallazgo	Dónde	Impacto	Severidad
4	El explorador del mapa reutiliza el selector de ubicación del formulario y su store	`LocationMap`, `stores/form/location`	Pin fijo en Quito en cualquier zona, tocar el mapa escribe en el store del formulario	Alta
5	Cero tests en el front y uno en el back	ambos repos	Nada avisa cuando un fix rompe otra cosa	Alta
6	Minimapa del modal "Zona no disponible" sin props	`ZoneUnavailableModal.tsx`	Siempre muestra Quito y ninguna zona dibujada	Media
7	Taxonomía quemada en el front	`SUBCATEGORY_MAP`, `constants/categories.ts`	Una subcategoría nueva en el admin no llega al formulario	Media
8	Landings de Ecuador y México codificadas aunque existe el CMS de países	`[country]/page.tsx:74-79`	El copy cambia con un deploy, no desde el admin	Media
9	Cuatro caminos distintos insertan lugares y siete helpers de auditoría	back, servicios	Cada camino valida distinto	Media
10	Sincronización de WiFi atada a Quito	`wifi-sync.service.js:3-9`	No escala a otra ciudad sin tocar código	Media

#	Hallazgo	Dónde	Impacto	Severidad
11	Código muerto: 26 archivos, ~2.300 líneas, y el endpoint `/places/banks`	ambos repos	Ruido para personas y LLMs	Baja

Mapa

Recomendación: no rehacerlo desde cero, sino reestructurar su núcleo (estado de zona y cámara) y conservar las piezas que funcionan: marcadores, tarjetas, iconos y sidebar. Los parches vienen de que nadie es dueño del estado; reescribir todo sin resolver eso repetiría el problema.

Por qué se acumulan parches

- **Cinco lugares mueven la cámara**: el efecto de `selectedPosition` (zoom 18), el efecto de `zoneTarget` (`fitBounds`), `handleOutsideZone` , `handleLocateRequest` y el fallback de `handleZoneSelect` . Cada fix de "la cámara salta" agrega otro ref o condición (`initialFlownRef` , `targetZoneHandledRef`).
- **La zona elegida vive en tres sitios** (estado, `?zone=` y localStorage) y se resuelve dos veces: en el servidor (`resolveZone`) y en el cliente, en el estado inicial y otra vez en un efecto.
- **El pack por defecto `1` está escrito en tres archivos**: `map/page.tsx:8` , `ViewMap.tsx:137` y `useSidebarState.ts:37` .
- **La lógica de zonas está repetida**: `activeZones` se filtra en cuatro lugares, la máscara gris se calcula dos veces y el centro de una zona se calcula de tres formas. Hay dos `isInsideAnyActiveZone` con resultados opuestos cuando no hay zonas (`useLocationMapLogic` devuelve `true` , `useMapZones` devuelve `false`).
- **Siete detecciones de móvil** con `innerWidth <= 768` . `ViewMap` trata un teléfono en horizontal como móvil y `LocationMap` no, y esta última no se actualiza al rotar.
- **El explorador reutiliza el selector de ubicación del formulario**: arranca en Quito porque usa el centro por defecto del store del formulario, deja el pin `GomiPointer` en Quito aunque estés en Cuenca (probablemente el "único marcador" del ticket de ayer), y tocar el mapa escribe en el store del formulario y muestra el aviso de perímetro.

- **Optimizaciones a medias**: el filtrado por viewport está apagado en móvil y las utilidades de clustering (`placeCluster.ts`) se escribieron y nunca se usaron.
- **Cuatro overlays hechos a mano** con `position: fixed` y `zIndex: 99999` (11 usos) en vez del `Modal` de Mantine. `MapPage` usa la clase `privacy-policy-page` , copiada de otra vista.

Estructura propuesta

Pieza	Hoy	Propuesta
Componentes	`LocationMap` sirve al explorador, al formulario y al admin	`MapExplorer` para `/map` y `LocationPicker` para formulario y admin, sobre un `MapCanvas` común
Estado de zona	`useState` en `ViewMap` + URL + localStorage	Un store del mapa: zona, pack y su sincronización con la URL en un solo lugar
Cámara	Cinco efectos y handlers con `flyTo` / `fitBounds`	Una "intención de cámara" en el store (encuadrar zona, ir al usuario, ir a un lugar) y un solo efecto que la ejecuta
Zonas	Hooks y cálculos repetidos	Un módulo `zones` con TanStack Query y funciones puras (`isInside` , `bbox` , `center` , `mask`), testeables
Móvil	Siete implementaciones	Un hook `useIsMobile`

Con esto, el bug del modal de ayer no habría existido: el modal solo habría podido "pedir encuadrar la zona X".

Valores quemados que deberían venir del admin

La regla propuesta: todo lo que cambia por ciudad, país o contenido vive en la base de datos, se edita en el admin y llega por API. En el código solo quedan valores técnicos. Estos casos la rompen hoy:

Valor	Dónde	Qué debería ser
`packId: 1` en el envío del formulario y en su esquema	front `collectFormData.ts:35`, `useFormSchema.ts:22`	El pack de la zona donde está el punto; el back lo valida contra el polígono
`SUBCATEGORY_MAP` con 6 ids fijos (basurero, baño, banca, wifi, cajero, gema)	front `lib/api.ts:976`	La lista de subcategorías del pack desde la API de taxonomía
Pack por defecto `1`	front `map/page.tsx:8`, `ViewMap.tsx:137`, `useSidebarState.ts:37`	Una zona por defecto marcada en el admin
Coordenadas de Quito como centro inicial	front `stores/form/location/location.ts:57`, `ZoneUnavailableMapContent.tsx:43`, `PlacePickerMapContent.tsx:21`	El centro o `bbox` de la zona activa
Alias de slugs (`iconics`, `iconicos`, `local-highlight`…), orden de secciones y color `#FFB703`	front `MapPage/constants/categories.ts`	Campos de la categoría en el admin: tipo de sección, orden y color
Landings de Ecuador y México	front `[country]/page.tsx:74-79`, textos en `messages/*.json`	El CMS de países que ya existe en el admin
Lista de 10 bancos ecuatorianos	back `place.service.js:333` (`/places/banks`, sin uso en el front)	Opciones de una propiedad en el admin, o borrar el endpoint
URL de WiFi de Quito, parroquia, pack 1, categoría 1, subcategoría 4	back `wifi-sync.service.js:3-9`	Una configuración de fuente externa por pack

Valor	Dónde	Qué debería ser
Usuario de sistema `system@gomiguide.com`	back `place-change-request.service.js:565`	Variable de entorno o semilla de la base de datos
URL de Stack Auth repetida	back `middlewares/auth.js:5`, `services/user.service.js:6`	Una sola constante en `config/`

El `map_zones.json` de respaldo también es una segunda fuente de verdad. Si la API de zonas falla, el mapa muestra zonas que pueden no coincidir con el admin. Conviene decidir si se mantiene.

Backend

Hay que corregir ya un caso de SQL armado con texto. Después, lo que más rinde es unificar la escritura de lugares en una sola función de dominio: hoy hay seis caminos que insertan lugares o requests.

Seguridad

- **Aprobar un change request arma el `UPDATE` con texto** (`place-change-request.service.js:363-375`): `status`, el JSON de `properties` y los ids van interpolados en `$executeRawUnsafe`. `POST /api/v1/place-change-requests` es público y acepta `update` y `delete` con `proposedProperties: z.record(z.any())`. Un valor con apóstrofe rompe la aprobación, y un valor malicioso se ejecuta cuando un admin aprueba. Solución: `prisma.place.update` o `Prisma.sql`, y decidir si los requests públicos de editar o borrar deben existir sin autenticación.
- **`getPlacesForMap` también interpola** `lang`, `packId` y coordenadas (`place.service.js:692-762`). Hoy es seguro solo porque zod limita `lang` a `es`/`en` y valida los números; un cambio en el validator lo abre. Debe pasar a parámetros.
- Conviven tres estilos de consulta: interpolación, parámetros `$1` (`admin-places`) y el ORM de Prisma. Hay que acordar uno: Prisma por defecto y `Prisma.sql` solo para PostGIS.

Seis caminos escriben lugares

Camino	Archivo	Qué escribe
Formulario público	`place.service.js:16`	Change request pendiente
Carga masiva, pendiente	`place.service.js:508`	Change request pendiente
Carga masiva, auto-aprobada	`place.service.js:561`	Lugar + change request aprobado
Crear desde el admin	`admin-places.service.js:15`	Lugar
Aprobar un change request	`place-change-request.service.js:285`	Lugar
Sincronización de WiFi	`wifi-sync.service.js:172`	Lugar

Cada camino valida propiedades, pack y coordenadas a su manera. Ninguno comprueba que el punto caiga dentro del polígono del pack. Propuesta: un único `placeWriter` (crear, actualizar, aprobar) que valide y audite, y que los seis caminos lo llamen.

Consultas y helpers repetidos

- El `SELECT` de lugares con los joins de traducción y el respaldo a `es` está copiado cuatro veces: mapa y detalle en `place.service.js`, lista y detalle en `admin-places.service.js`. Es justo tu sospecha: una sola consulta con filtros (pack, id, bbox, estado) reemplaza a las cuatro. No medí rendimiento (sin `EXPLAIN`); eso queda pendiente.
- Siete helpers de auditoría con dos firmas distintas (`_audit` en taxonomía, países, propiedades y zonas; `_createAuditLog` en lugares, change requests y WiFi). Van a un único `utils/audit.js`.
- `place.service.js` (984 líneas) mezcla mapa público, formulario, carga masiva y la lista de bancos. `admin-properties.service.js` tiene 940.

Base técnica

- Un solo test (`utils/place-card.test.js`).

- Es JavaScript sin tipos: el JSDoc es la única información de tipos, así que no conviene borrarlo en la limpieza de comentarios. La solución definitiva es migrar a TypeScript archivo por archivo con `allowJs`.

Código muerto

En el front hay 26 archivos (~2.300 líneas) que ninguna ruta de Next importa. Se detectaron recorriendo el grafo de imports desde `src/app` y se confirmaron buscando cada nombre en el resto del código. Se pueden borrar en un solo PR.

Grupo	Archivos	Líneas
Home vieja y su copia de respaldo	`views/HomePage/*`, `views/HomePage-backup/*` y lo que solo ellas usan (`ThemeDemo`, `ColorSchemeToggle`, `LanguageToggle`)	~630
Formulario viejo	`FormPage/sections/LocationSection.tsx`, `Open.tsx`, `components/form/DynamicFormSection.tsx`, `DynamicFormField.tsx`, `hooks/useSubmitPlace.ts`, `stores/form/openingHours.ts`, `pricing.ts`, `location/validators.ts`	~1.150
Mapa	`utils/placeCluster.ts`, `utils/radialPosition.ts`, `SideBar/hooks/useBottomSheet.ts`, `constants/layout.constants.ts`	~400
Sueltos	`hooks/useCountdown.ts`, `lib/performance.ts`, `stores/data/exampleStore.ts`	~115

Dentro de archivos vivos también hay exports sin uso: `getNearbyPlaces` (`lib/map.ts`), `usePlace` (`usePlaces.ts`), `getHiddenGemsBadgeSvgs`, `DYNAMIC_PIN_FALLBACK_COLOR`, `LOCAL_HIGHLIGHTS_SLUG`, y los métodos `api.getPlace`, `api.getUser`, `api.getPropertyDefinition` y `api.deleteCountryLanding`. `layout.constants.ts` está muerto mientras `ViewMap` redefine esas mismas constantes.

En el back: `/places/banks` sin consumidor y rutas comentadas en `place-change-requests.routes.js`.

Para que no vuelva a pasar: correr Knip en CI (detecta archivos, exports y dependencias sin uso) y no guardar copias tipo `-backup`, porque git ya guarda el historial.

Base para Gomi B2B

Recomendación: un monolito modular en cada repo, con módulos por dominio y fronteras claras. Microservicios o micro-frontends solo cuando un módulo tenga su propio equipo, su propia carga o su propio ritmo de despliegue. Con fronteras limpias, extraer un módulo después es cambiar llamadas a funciones por llamadas HTTP, sin romper lo demás.

Módulo	Backend	Frontend
Lugares	`placeWriter` + una consulta con filtros	Mapa (explorador) y formulario
Taxonomía	Categorías, subcategorías, propiedades	Lo consume todo lo demás; nada se quema en el front
Zonas / packs	Polígonos, estado, zona por defecto	Módulo `zones` único
Contenido	Países y landings	Una sola landing genérica alimentada por el CMS
Integraciones	WiFi y futuras fuentes, configuradas por pack	—
Auditoría	`utils/audit.js`	—
B2B	Nuevo módulo	Nuevo módulo en `src/features/` (hoy vacía)

Reglas que hacen posible la migración escalonada:

- Un módulo solo toca sus tablas a través de su servicio; los demás usan sus funciones públicas.
- Un contrato compartido entre front y back (OpenAPI o esquemas zod compartidos). Hoy `lib/api.ts` tiene 1.611 líneas tipadas a mano que pueden desalinearse del back sin que nada avise.
- El admin y la app pública tienen usuarios y ritmos distintos. Son el primer candidato natural a separarse si algún día hacen falta micro-frontends; por ahora basta con mantenerlos en módulos separados.

Preguntas abiertas para la reunión:

- ¿Quién es un cliente B2B (municipio, empresa, operador turístico) y qué controla: sus propios packs, sus propios puntos, solo reportes?
- ¿El pack es la frontera de cliente (tenant) o hace falta una entidad "organización" por encima? Hoy el pack es lo más parecido.

Orden de trabajo propuesto

Cinco fases, cada una deja el código mejor que la anterior sin bloquear el trabajo de producto. Regla para todas: cada PR de refactor trae tests de lo que toca.

Fase 0 — Urgente

- [ ] Pasar a parámetros el `UPDATE` de aprobación de change requests y `getPlacesForMap`
- [ ] Decidir si los change requests públicos de editar o borrar requieren autenticación
- [ ] Enviar el pack real desde el formulario y validar en el back que el punto cae en el polígono del pack

Fase 1 — Contexto y red de seguridad

- [ ] `AGENTS.md` actualizado en front y back (hoy el `CLAUDE.md` dice Leaflet y el mapa es Mapbox)
- [ ] Skills compartidas: commit, PR, code review y pruebas locales
- [ ] Borrar el código muerto y agregar Knip al CI
- [ ] Primeros tests: funciones de zonas en el front, `buildPlaceCard` y validaciones en el back

Fase 2 — Mapa

- [ ] Separar `MapExplorer` y `LocationPicker` sobre un `MapCanvas` común
- [ ] Store del mapa con zona, pack e intención de cámara; un solo efecto que mueve la cámara
- [ ] Módulo `zones` único y hook `useIsMobile`
- [ ] Modales con el `Modal` de Mantine; minimapa del modal con la zona real

Fase 3 — Backend y orden de archivos

- [ ] `placeWriter` único para los seis caminos
- [ ] Una consulta de lugares con filtros en vez de cuatro copias
- [ ] `utils/audit.js` y convención de SQL (Prisma + `Prisma.sql` solo para PostGIS)

- [ ] Partir los archivos de más de 800 líneas; separar estilos, tipos y constantes; traducciones por namespace
- [ ] Migración gradual del back a TypeScript

Fase 4 — El admin como única fuente de verdad

- [ ] El formulario usa la taxonomía del pack, no `SUBCATEGORY_MAP`
- [ ] Zona por defecto, orden y tipo de sección de las categorías editables en el admin
- [ ] Landing genérica para Ecuador y México desde el CMS de países
- [ ] Configuración de fuentes externas (WiFi) por pack
- [ ] Decidir si se mantiene `map_zones.json` como respaldo

Fase 5 — B2B

- [ ] Responder las preguntas abiertas sobre cliente y tenant
- [ ] Diseñar el módulo B2B sobre los módulos de las fases anteriores